[LABORATÓRIO DE] ESTRUTURA DE DADOS 2026.2

OBS.: Esta atividade pode ser realizada em grupo (até quatro integrantes).
Preencher planilha: link

ATIVIDADE DA UNIDADE 1:

Análise de Algoritmos de Busca e Ordenação

Objetivo: Desenvolver habilidades de implementação e análise de desempenho de algoritmos de busca e ordenação em Java, exclusivamente com arrays de inteiros (`int[]`), de modo a combinar teoria (complexidade assintótica) e prática experimental (tempos medidos), destacando cenários em que cada algoritmo é mais vantajoso. Os valores dos arrays deverão ser inteiros pertencentes ao intervalo [1, 100.000], sendo permitida a ocorrência de valores repetidos.

Tarefas Específicas:

1. **Implementação e Utilização de Algoritmos de Ordenação:**
    - Implemente:
        - **BubbleSort** (versão do *slide* **E** versão otimizada)
        - **SelectionSort** (versão do *slide* **E** versão estável)
        - **InsertionSort**
        - **MergeSort** (versão clássica)
        - **QuickSort** (versão do *slide* **E** versão do *slide*+*shuffle*). Utilize também `Arrays.sort(int[])` como implementação de referência da biblioteca padrão Java para comparação de desempenho. Não é necessário implementar essa versão!
        - **CountingSort**
    - Ordene arrays de inteiros.
2. **Implementação dos Algoritmos de Busca:**
    - Implemente (considerando vetor previamente ordenado):
        - **Busca Linear Iterativa**
        - **Busca Linear Recursiva**
        - **Busca Binária Iterativa**
        - **Busca Binária Recursiva**
        - **Busca Linear Iterativa Duas Pontas**
    - Realize os experimentos de busca considerando a busca pelo elemento -1, que não pertence ao intervalo de valores dos arrays e, portanto, está ausente.
3. **Análise de Performance:**
    - Meça tempos de execução com `System.nanoTime()` em três tamanhos de vetor (por exemplo, 50.000, 150.000 e 500.000 elementos; ajuste os tamanhos caso o tempo de execução ou a memória disponível inviabilizem os experimentos, mantendo os mesmos tamanhos para todos os algoritmos de ordenação que forem comparados diretamente).

- Compare algoritmos de ordenação em quatro cenários (vetores aleatórios, vetores já ordenados, vetores em ordem inversa, e vetores parcialmente ordenados). Compare também a eficiência entre buscas lineares e binárias.
- Para o cenário de vetores parcialmente ordenados, considere um vetor inicialmente ordenado e selecione aleatoriamente aproximadamente 15% das posições, embaralhando apenas os elementos dessas posições.
- Para cada combinação de algoritmo, tamanho e cenário, realizar pelo menos 25 execuções, descartando as 5 primeiras como *warm-up*, para minimizar a variabilidade da JVM. Os tempos das execuções de *warm-up* não devem ser utilizados no cálculo das estatísticas reportadas.
- Não use como entrada de um algoritmo a saída já ordenada de outro (isso distorce os resultados). Para cada combinação de tamanho e cenário, todos os algoritmos devem receber cópias da mesma entrada original.
- Após cada execução, verifique a correção do resultado, garantindo que o vetor esteja ordenado. Os tempos de execução da verificação não devem ser incluídos na medição do algoritmo.

4. **Relatório de Análise:**
   - Elabore um breve relatório em PDF contendo:
     - Metodologia dos testes (número de execuções, tamanhos dos vetores e cenários avaliados).
     - Tabelas e gráficos com os tempos de execução obtidos. *(Para facilitar o entendimento, veja as tabelas das questões 13 e 16 da Lista de Exercícios.)*
     - Síntese dos resultados obtidos à luz da complexidade assintótica esperada, discutindo eventuais divergências entre teoria e prática observadas nos experimentos.
5. **Vídeo de Apresentação:**
   - Deve-se entregar um vídeo, com participação de todos os integrantes, aparecendo imagem/vídeo do integrante. Sugiro todos gravarem com a câmera ligada (pode ser a do celular) em uma chamada via Google Meet.
   - O vídeo deve apresentar os principais resultados e responder às seguintes questões:
     - Os tempos observados foram compatíveis com os custos assintóticos esperados para cada algoritmo? Cite exemplos.
     - Em quais cenários (ordenado, aleatório, inverso, parcialmente ordenado) as diferenças entre os algoritmos ficaram mais evidentes?
     - Quais algoritmos de ordenação são estáveis? Explique por que essa característica pode ser importante.
     - Quais algoritmos utilizam memória auxiliar significativa? Como o consumo de memória influencia a escolha de um algoritmo de ordenação?
     - Se fosse necessário ordenar um grande conjunto de dados em uma aplicação real, quais algoritmos vocês escolheriam ou evitariam? Em que situações utilizariam uma implementação própria e em que situações utilizariam `Arrays.sort()`?
   - Não apresente todas as tabelas e todos os gráficos no vídeo! O foco deve estar na interpretação dos resultados e não na simples leitura dos dados.

Padronização da Entrega:

- **Formato dos Arquivos:** Crie uma pasta no Google Drive contendo o vídeo da apresentação e o relatório em PDF, no qual deve haver um link do repositório no GitHub (em vez do upload do vídeo, pode colocar no documento também o link do vídeo no YouTube).
- **Compartilhamento:** A pasta no Google Drive deve estar privada, compartilhada apenas com os integrantes do grupo, o professor (janderson@servidor.uepb.edu.br) e os monitores (agra.neto@aluno.uepb.edu.br, araujo.victor@aluno.uepb.edu.br, cruz.leticia@aluno.uepb.edu.br, j.caetano@aluno.uepb.edu.br, rodrigo.loiola@aluno.uepb.edu.br, rui.fernando@aluno.uepb.edu.br). O link da pasta deve estar na planilha.

Prazo de Entrega:

- **Prazo regular:** 31/10/2025 às 23h59min. Após esse período, a nota será zero.
- **Reposição:** dado o prazo de vários dias para a entrega do projeto, *a reposição da unidade ficará restrita às atividades presenciais realizadas em sala de aula (prova/minitestes).*